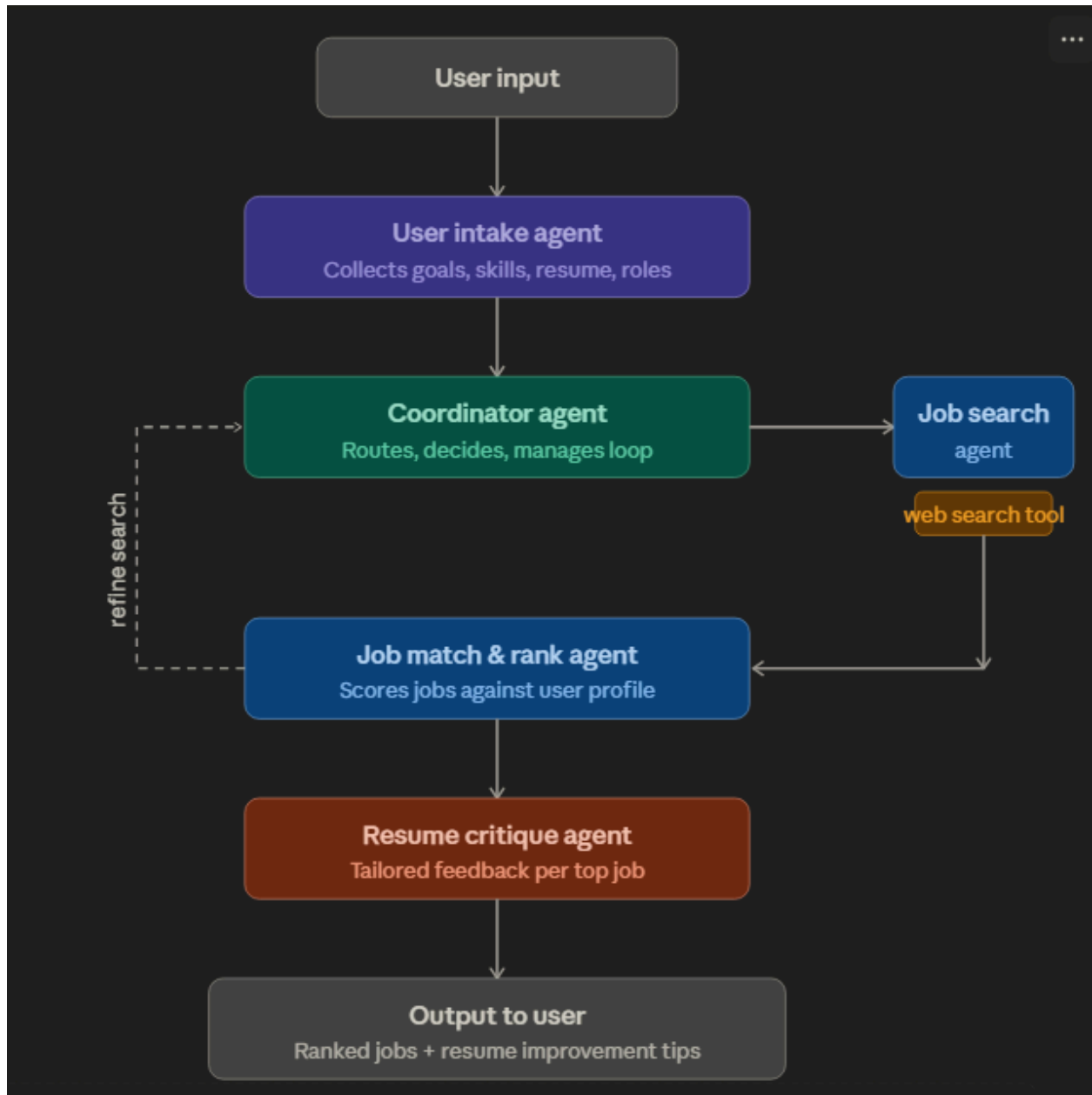


# Agentic Workflow Diagram



Here's a conceptual breakdown of how the workflow would look, then I'll render the diagram.

The workflow uses a **ReAct + Critique hybrid** pattern with three main phases: intake, search/match, and feedback. Here's what each agent does:

**User Intake Agent** — Collects the user's job preferences, skills, resume/CV, and desired roles. Structures this into a profile for downstream agents.

**Job Search Agent** — Uses a web search tool to scrape/search job boards (LinkedIn, Indeed, etc.) based on the user profile. Returns a raw list of relevant postings.

**Job Match & Rank Agent** — Scores and filters the job results against the user's profile. Eliminates irrelevant matches, ranks the rest by fit.

**Resume Critique Agent** — For each top job, analyzes the user's resume/CV against the job description and generates tailored feedback and suggestions.

**Coordinator Agent** — Orchestrates the loop: decides whether to re-search with refined parameters (ReAct loop) or pass results to the critique stage. Acts as the "brain" of the pipeline.

v

visualize show\_widget

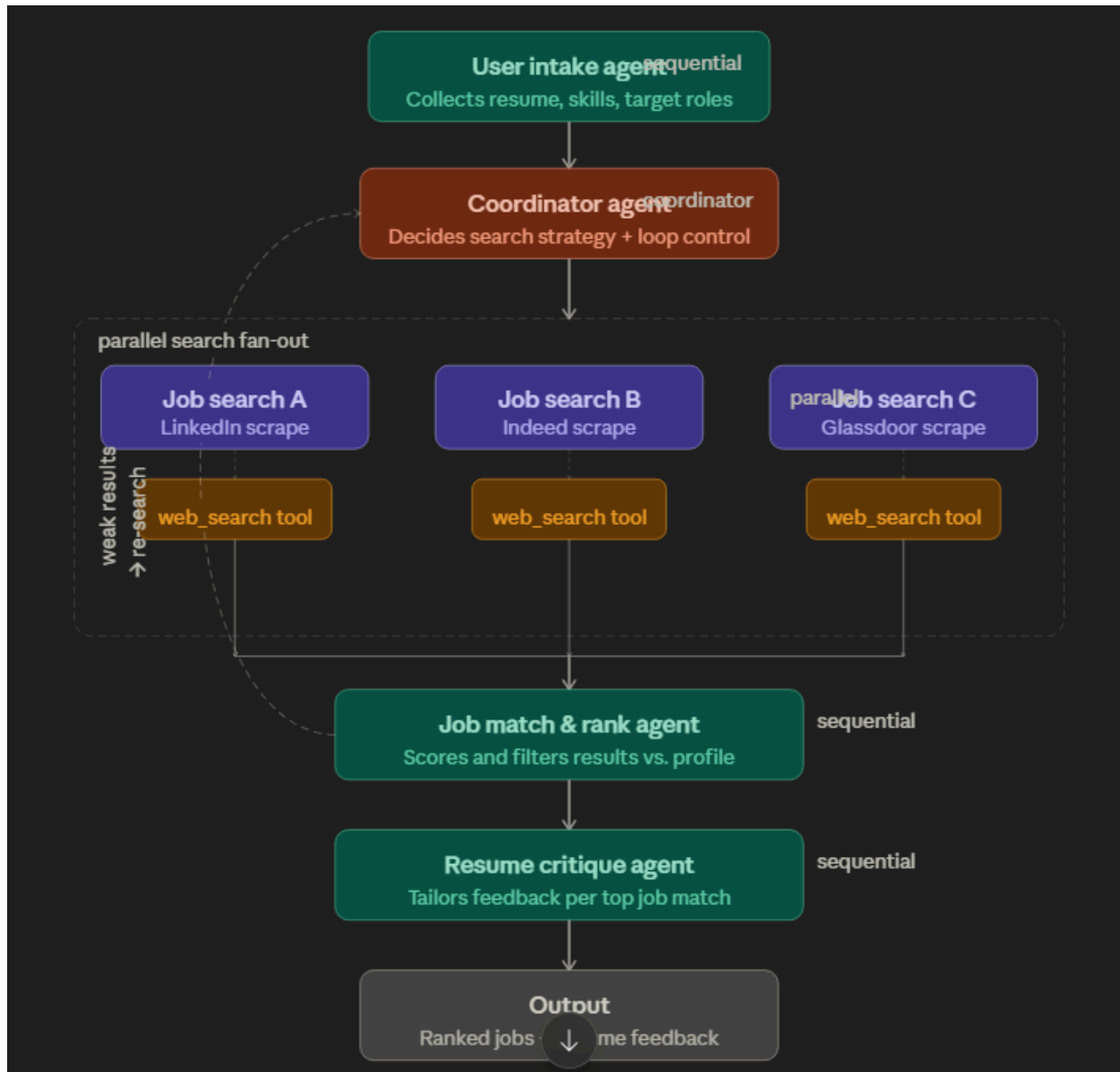
Here's a summary of the full conceptual model:

**Five agents total**, which satisfies the "at least 3 regular agents" requirement:

1. **User Intake Agent** — Structured input collection (skills, resume, target roles, location, etc.)
2. **Coordinator Agent** — The "brain" that decides whether results are good enough or need another search loop
3. **Job Search Agent** — Fires the web search tool against job boards, returns raw postings
4. **Job Match & Rank Agent** — Filters and scores results against the user profile
5. **Resume Critique Agent** — Reads the top job descriptions and generates specific resume feedback

**Design patterns used:**

- **ReAct loop** — The coordinator can send the match agent's results back for a refined search if the results are weak (the dashed loop on the left)
- **Sequential flow** — Intake → Coordinator → Search → Match → Critique → Output
- **Tool use** — The Job Search Agent uses a web search tool (your SpiderAI integration point)



^^^ we decided on using this one

<http://spiderweb.richmond.edu:11076/>

# Agent Details

# Regular Agents (the 4 worker agents)

---

## Agent 1

- **Name:** `user_intake_agent`
  - **Model:** gemini-2.5-flash
  - **Description:** Collects and structures the user's job search profile including their resume, skills, experience level, target job titles, preferred location, and any other preferences needed for downstream agents to perform an effective search.
  - **Instructions:** Ask the user for the following information and structure it into a clean profile: (1) Their current resume or CV text, (2) Key skills and technologies, (3) Target job titles or roles, (4) Preferred location or remote preference, (5) Years of experience, (6) Any industries or companies to include or exclude. Once collected, output a structured JSON-style profile summary with keys: `resume_text`, `skills`, `target_roles`, `location`, `experience_years`, `preferences`. This output will be passed to downstream agents.
  - **File:** No file (or optionally accept resume upload)
  - **Tool:** No Tool
  - **Output Key:** `user_profile`
- 

## Agent 2 — LinkedIn Search

- **Name:** `job_search_linkedin`
  - **Model:** gemini-2.5-flash
  - **Description:** Searches LinkedIn job postings using the user profile to find relevant open positions. Returns a raw list of job postings including title, company, location, and job description snippet.
  - **Instructions:** Using the `user_profile` provided, construct targeted search queries for LinkedIn job postings. Search for jobs matching the user's `target_roles`, `skills`, and location preferences. For each result, extract: job title, company name, location, job description summary, required skills, and job URL. Return a list of up to 10 relevant job postings in structured format. Focus only on active, current postings.
  - **File:** No file
  - **Tool:** Tool (`web_search`)
  - **Output Key:** `linkedin_jobs`
- 

## Agent 3 — Indeed Search

- **Name:** `job_search_indeed`

- **Model:** gemini-2.5-flash
  - **Description:** Searches Indeed job postings using the user profile to find relevant open positions. Returns a raw list of job postings including title, company, location, and job description snippet.
  - **Instructions:** Using the user\_profile provided, construct targeted search queries for Indeed job postings. Search for jobs matching the user's target\_roles, skills, and location preferences. For each result, extract: job title, company name, location, job description summary, required skills, and job URL. Return a list of up to 10 relevant job postings in structured format. Focus only on active, current postings.
  - **File:** No file
  - **Tool:** Tool (web\_search)
  - **Output Key:** indeed\_jobs
- 

#### Agent 4 — Glassdoor Search

- **Name:** job\_search\_glassdoor
  - **Model:** gemini-2.5-flash
  - **Description:** Searches Glassdoor job postings using the user profile to find relevant open positions. Returns a raw list of job postings including title, company, location, salary range if available, and job description snippet.
  - **Instructions:** Using the user\_profile provided, construct targeted search queries for Glassdoor job postings. Search for jobs matching the user's target\_roles, skills, and location preferences. For each result, extract: job title, company name, location, salary range (if listed), job description summary, required skills, and job URL. Return a list of up to 10 relevant job postings in structured format. Focus only on active, current postings.
  - **File:** No file
  - **Tool:** Tool (web\_search)
  - **Output Key:** glassdoor\_jobs
- 

#### Agent 5 — Job Match & Rank

- **Name:** job\_match\_agent
- **Model:** gemini-2.5-flash
- **Description:** Receives the merged list of job postings from all search agents and the user profile, then scores and ranks each job by relevance. Filters out poor matches and returns the top 5 ranked jobs with match reasoning.
- **Instructions:** You will receive a combined list of job postings from linkedin\_jobs, indeed\_jobs, and glassdoor\_jobs, along with the user\_profile. For each job posting, score it from 1–10 based on: skill alignment (40%), role title match (30%), location/remote fit (20%), and experience level match (10%). Remove any jobs scoring below 5. Return the top 5 highest-scoring jobs in ranked order, each with: job title,

company, location, URL, match score, and a 2–3 sentence explanation of why this job is a strong match. Also output a field called results\_quality rated low/medium/high based on overall match scores.

- **File:** No file
  - **Tool:** No Tool
  - **Output Key:** ranked\_jobs
- 

## Agent 6 — Resume Critique

- **Name:** resume\_critique\_agent
  - **Model:** gemini-2.5-flash
  - **Description:** Takes the user's resume and the top ranked job matches, then generates specific, actionable resume and CV improvement suggestions tailored to each job description.
  - **Instructions:** You will receive the user\_profile (which contains resume\_text) and ranked\_jobs (the top job matches). For each of the top 3 jobs in ranked\_jobs, compare the user's resume against the job description and generate: (1) A list of keywords or skills missing from the resume that appear in the job description, (2) Specific bullet point rewrites or additions to strengthen the resume for that job, (3) A recommended resume summary or objective tailored to that role, (4) An overall readiness score from 1–10 for that job. Return results as a structured list, one feedback block per job.
  - **File:** No file
  - **Tool:** No Tool
  - **Output Key:** resume\_feedback
- 

## Sequential Agent

- **Name:** post\_search\_pipeline
  - **Description:** Runs the job match and resume critique agents in sequence after all parallel searches have completed. First ranks the merged job results, then generates resume feedback based on the top matches.
  - **Agents to run in sequence:** job\_match\_agent, resume\_critique\_agent
- 

## Parallel Agent

- **Name:** parallel\_job\_search



- **Description:** Runs all three job board search agents simultaneously — LinkedIn, Indeed, and Glassdoor — to collect job postings in parallel and reduce total search time.
  - **Agents to run in parallel:** `job_search_linkedin`, `job_search_indeed`, `job_search_glassdoor`
- 

## Loop Agent

- **Name:** `search_refinement_loop`
  - **Description:** Loops the parallel job search and job match steps together. If the coordinator determines results quality is low, this loop re-runs the search with refined parameters until quality is acceptable or max iterations is reached.
  - **Agents to run in sequence then loop:** `parallel_job_search`, `job_match_agent`
  - **Maximum iterations:** 3
- 

## Coordinator Agent

- **Name:** `search_coordinator`
  - **Description:** Orchestrates the overall job search workflow. Receives the user profile, triggers the search refinement loop, evaluates results quality from the match agent, and decides whether to continue to resume critique or loop back for a better search. Acts as the central brain of the pipeline.
  - **Subagents:** `search_refinement_loop`, `resume_critique_agent`
- 

## Root Agent

- **Name:** `user_intake_agent`
- 

## Summary of the full agent tree

```
root → user_intake_agent
      ↓
      search_coordinator (coordinator)
      ↓
```

```
search_refinement_loop (loop, max 3)
├─ parallel_job_search (parallel)
│   ├── job_search_linkedin
│   ├── job_search_indeed
│   └─ job_search_glassdoor
└─ job_match_agent
    ↓ (if results_quality = high/medium)
resume_critique_agent
```

The root is `user_intake_agent` because it's the entry point that collects everything before handing off to the coordinator. The coordinator then drives the loop and decides when quality is good enough to proceed to critique.

Agent Code

## REGULAR AGENTS CODES:

### User Intake agent

```
#
# Code to create agent: user_intake_agent
#

# No file information to insert in the query
instruction_insert = "

# Code the for agent itself
user_intake_agent = LlmAgent(
    name=""user_intake_agent"",
    model=""gemini-2.5-flash"",
    description="" Collects and structures the user's job search profile including their resume,
skills, experience level, target job titles, preferred location, and any other preferences needed
for downstream agents to perform an effective search.""",
    instruction=""Ask the user for the following information and structure it into a clean profile: (1)
Their current resume or CV text, (2) Key skills and technologies, (3) Target job titles or roles, (4)
Preferred location or remote preference, (5) Years of experience, (6) Any industries or
companies to include or exclude. Once collected, output a structured JSON-style profile
summary with keys: resume_text, skills, target_roles, location, experience_years, preferences.
This output will be passed to downstream agents.
"" + instruction_insert,
    output_key = ""user_profile"",
)
)
```

### Linkedin job search

```
#
# Code to create agent: job_search_linkedin
#

# No file information to insert in the query
instruction_insert = "

# Code the for agent itself
job_search_linkedin = LlmAgent(
    name=""job_search_linkedin"",
    model=""gemini-2.5-flash"",
```

```

        description="""Searches LinkedIn job postings using the user profile to find relevant open
positions. Returns a raw list of job postings including title, company, location, and job description
snippet."""
        instruction="""Using the user_profile provided, construct targeted search queries for LinkedIn
job postings. Search for jobs matching the user's target_roles, skills, and location preferences.
For each result, extract: job title, company name, location, job description summary, required
skills, and job URL. Return a list of up to 10 relevant job postings in structured format. Focus
only on active, current postings.
""" + instruction_insert,
        output_key = """linkedin_jobs"""
    )

```

### Indeed job search

```

#
# Code to create agent: job_search_indeed
#

# No file information to insert in the query
instruction_insert = ""

# Code the for agent itself
job_search_indeed = LlmAgent(
    name="""job_search_indeed""",
    model="""gemini-2.5-flash""",
    description="""Searches Indeed job postings using the user profile to find relevant open
positions. Returns a raw list of job postings including title, company, location, and job description
snippet.""",
    instruction="""Using the user_profile provided, construct targeted search queries for Indeed
job postings. Search for jobs matching the user's target_roles, skills, and location preferences.
For each result, extract: job title, company name, location, job description summary, required
skills, and job URL. Return a list of up to 10 relevant job postings in structured format. Focus
only on active, current postings.""" + instruction_insert,
    output_key = """indeed_jobs""",
)

```

### Glassdoor job search

```

#
# Code to create agent: job_search_glassdoor
#

# No file information to insert in the query
instruction_insert = "

# Code the for agent itself
job_search_glassdoor = LlmAgent(
    name=""job_search_glassdoor"",
    model=""gemini-2.5-flash"",
    description=""Searches Glassdoor job postings using the user profile to find relevant open
positions. Returns a raw list of job postings including title, company, location, salary range if
available, and job description snippet.""",
    instruction=""Using the user_profile provided, construct targeted search queries for
Glassdoor job postings. Search for jobs matching the user's target_roles, skills, and location
preferences. For each result, extract: job title, company name, location, salary range (if listed),
job description summary, required skills, and job URL. Return a list of up to 10 relevant job
postings in structured format. Focus only on active, current postings.
"" + instruction_insert,
    output_key = ""glassdoor_jobs"",
)

```

### Job match list

```

#
# Code to create agent: job_match_agent
#

# No file information to insert in the query
instruction_insert = "

# Code the for agent itself
job_match_agent = LlmAgent(
    name=""job_match_agent"",
    model=""gemini-2.5-flash"",
    description=""Receives the merged list of job postings from all search agents and the user
profile, then scores and ranks each job by relevance. Filters out poor matches and returns the
top 5 ranked jobs with match reasoning.""",

```

```

instruction="""You will receive a combined list of job postings from linkedin_jobs, indeed_jobs,
and glassdoor_jobs, along with the user_profile. For each job posting, score it from 1–10 based
on: skill alignment (40%), role title match (30%), location/remote fit (20%), and experience level
match (10%). Remove any jobs scoring below 5. Return the top 5 highest-scoring jobs in ranked
order, each with: job title, company, location, URL, match score, and a 2–3 sentence
explanation of why this job is a strong match. Also output a field called results_quality rated
low/medium/high based on overall match scores.""" + instruction_insert,
    output_key = """ranked_jobs""" ,

)

```

### Resume feedback

```

#
# Code to create agent: resume_critique_agent
#

# No file information to insert in the query
instruction_insert = "

# Code the for agent itself
resume_critique_agent = LlmAgent(
    name="""resume_critique_agent""",
    model="""gemini-2.5-flash""",
    description="""Takes the user's resume and the top ranked job matches, then generates
specific, actionable resume and CV improvement suggestions tailored to each job
description.""" ,
    instruction="""You will receive the user_profile (which contains resume_text) and ranked_jobs
(the top job matches). For each of the top 3 jobs in ranked_jobs, compare the user's resume
against the job description and generate: (1) A list of keywords or skills missing from the resume
that appear in the job description, (2) Specific bullet point rewrites or additions to strengthen the
resume for that job, (3) A recommended resume summary or objective tailored to that role, (4)
An overall readiness score from 1–10 for that job. Return results as a structured list, one
feedback block per job.
""" + instruction_insert,
    output_key = """resume_feedback""" ,

)

```

### **Sequential Agent**

```
#  
# Code to create agent: post_search_pipeline  
#  
post_search_pipeline = SequentialAgent(  
    name=""post_search_pipeline"",  
    description=""Runs the job match and resume critique agents in sequence after all parallel  
searches have completed. First ranks the merged job results, then generates resume feedback  
based on the top matches."",  
    sub_agents=[job_match_agent, resume_critique_agent],  
)
```

### **Parallel Agent**

```
#  
# Code to create agent: parallel_job_search  
#  
parallel_job_search = ParallelAgent(  
    name=""parallel_job_search"",  
    description=""Runs all three job board search agents simultaneously — LinkedIn, Indeed,  
and Glassdoor — to collect job postings in parallel and reduce total search time."",  
    sub_agents=[job_search_linkedin, job_search_indeed, job_search_glassdoor],  
)
```

### **Loop Agent**

```
#
```



```
# Code to create agent: search_refinement_loop
#
search_refinement_loop = LoopAgent(
    name=""search_refinement_loop"",
    description=""Loops the parallel job search and job match steps together. If the coordinator
determines results quality is low, this loop re-runs the search with refined parameters until
quality is acceptable or max iterations is reached."",
    sub_agents=[parallel_job_search, job_match_agent],
    max_iterations=3
)
```

### **Coordinator Agent**

```
#
# Code to create agent: search_coordinator
#
search_coordinator = Agent(
    name=""search_coordinator"",
    description=""Orchestrates the overall job search workflow. Receives the user profile,
triggers the search refinement loop, evaluates results quality from the match agent, and decides
whether to continue to resume critique or loop back for a better search. Acts as the central brain
of the pipeline."",
    sub_agents=[search_refinement_loop, resume_critique_agent],
)
```

### **Root Agent**

```
root_agent = user_intake_agent
```

Final Code

```

#####
# Import the appropriate packages
#####
import warnings
warnings.filterwarnings("ignore")

!pip install markdown

import asyncio
import os
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.agents import Agent, LlmAgent, SequentialAgent,
ParallelAgent, LoopAgent
from google.adk.tools import google_search
from google.adk.code_executors import BuiltInCodeExecutor
from google.adk.tools.tool_context import ToolContext
from google.adk.agents.callback_context import CallbackContext
from google.genai import types
from google import genai
from markdown import Markdown

#####
# Setup the API Key
#####
api_key = "INSERT API CODE HERE "
os.environ["GOOGLE_API_KEY"] = api_key

if "GOOGLE_GENAI_USE_VERTEXAI" in os.environ:
    del os.environ["GOOGLE_GENAI_USE_VERTEXAI"]

#####
# Setup the memory
#####
session_service = InMemorySessionService()

#####
# Setup the exit_tool for Loop Agents
#####

```

```

# Exit loop tool
def exit_loop(tool_context: ToolContext):
    print(f" [Tool Call] exit_loop triggered by
{tool_context.agent_name}")
    tool_context.actions.escalate = True
    tool_context.actions.skip_summarization = True
    return {}

#####
# Hypothetical Tools -- ADD CODE FOR ANY HYPOTHETICAL TOOLS
#####

#####
# Agents -- ADD CODE FOR ALL THE AGENTS BELOW
#####

# -----
# REGULAR AGENTS
# -----

instruction_insert = ""

user_intake_agent = LlmAgent(
    name="user_intake_agent",
    model="gemini-2.5-flash",
    description=(
        "Collects and structures the user's job search profile including "
        "resume text, skills, target roles, preferred location, experience,
"
        "and other preferences needed for downstream agents."
    ),
    instruction="""
You are the intake agent for a job search and resume refinement workflow.

Your job is to collect and structure the user's profile using these
fields:
1. resume_text
2. skills
3. target_roles

```

4. location
5. experience\_years
6. preferences

Treat these as MAJOR required fields:

- resume\_text
- skills
- target\_roles
- location
- experience\_years

Treat preferences as a MINOR field:

- industries or companies to include or exclude
- remote preference details beyond what is already stated
- other optional job search constraints

Rules:

- If a MAJOR field is missing, ask a short follow-up question to get it.
- If only MINOR preference information is missing, make a reasonable default assumption and continue.
- Do not stop the workflow just because minor preference details are missing.
- Be concise and practical.

If you need to make a default assumption for preferences, use:

- include\_industries: []
- exclude\_industries: []
- include\_companies: []
- exclude\_companies: []
- remote\_preference: "open to roles consistent with the user's stated location preference"

After collecting enough information, output ONLY a clean structured profile with these keys:

- resume\_text
- skills
- target\_roles
- location
- experience\_years
- preferences

Return no extra explanation outside the structured profile.

```
""",
    output_key="user_profile",
)
```

```
job_search_linkedin = LlmAgent(
    name="job_search_linkedin",
    model="gemini-2.5-flash",
    description=(
        "Searches LinkedIn job postings using the user profile to find
relevant open positions."
    ),
    instruction="""
```

Using the user\_profile, search for relevant LinkedIn job postings that match:

- target roles
- key skills
- preferred location or remote preference

Return up to 10 current and relevant jobs.

For each job include:

- job\_title
- company\_name
- location
- job\_description\_summary
- required\_skills
- job\_url
- source

Only return jobs that appear current or active.

Set source to LinkedIn.

Do not invent job postings or URLs.

```
"" + instruction_insert,
    tools=[google_search],
    output_key="linkedin_jobs",
)
```

```
job_search_indeed = LlmAgent(
    name="job_search_indeed",
```

```

    model="gemini-2.5-flash",
    description=(
        "Searches Indeed job postings using the user profile to find
relevant open positions."
    ),
    instruction="""
Using the user_profile, search for relevant Indeed job postings that
match:
- target roles
- key skills
- preferred location or remote preference

Return up to 10 current and relevant jobs.
For each job include:
- job_title
- company_name
- location
- job_description_summary
- required_skills
- job_url
- source

Only return jobs that appear current or active.
Set source to Indeed.
Do not invent job postings or URLs.
""" + instruction_insert,
    tools=[google_search],
    output_key="indeed_jobs",
)

job_search_glassdoor = LlmAgent(
    name="job_search_glassdoor",
    model="gemini-2.5-flash",
    description=(
        "Searches Glassdoor job postings using the user profile to find
relevant open positions."
    ),
    instruction="""
Using the user_profile, search for relevant Glassdoor job postings that
match:

```

- target roles
- key skills
- preferred location or remote preference

Return up to 10 current and relevant jobs.

For each job include:

- job\_title
- company\_name
- location
- salary\_range\_if\_available
- job\_description\_summary
- required\_skills
- job\_url
- source

Only return jobs that appear current or active.

Set source to Glassdoor.

Do not invent job postings, salary ranges, or URLs.

```
""" + instruction_insert,
    tools=[google_search],
    output_key="glassdoor_jobs",
)
```

```
job_match_agent = LlmAgent(
    name="job_match_agent",
    model="gemini-2.5-flash",
    description=(
        "Receives job postings from all search agents and the user profile,"
        "then scores, filters, and ranks the jobs by relevance."
    ),
    instruction="""
```

You will receive:

- user\_profile
- linkedin\_jobs
- indeed\_jobs
- glassdoor\_jobs

First merge all jobs into one combined list.



Then remove obvious duplicates using title, company, location, and URL similarity.

For each remaining job, assign a score from 1 to 10 using:

- skill alignment: 40%
- role title match: 30%
- location or remote fit: 20%
- experience level match: 10%

Remove any jobs scoring below 5.

Return the top 5 highest-scoring jobs in ranked order.

For each ranked job include:

- rank
- job\_title
- company
- location
- source
- url
- match\_score
- match\_explanation

Also return:

- average\_match\_score
- results\_quality

Set results\_quality as:

- high if average\_match\_score >= 8
- medium if average\_match\_score >= 6 and < 8
- low if average\_match\_score < 6

Return the output in a structured format only.

```
""" + instruction_insert,  
    output_key="ranked_jobs",  
)
```

```
search_refinement_agent = LlmAgent(  
    name="search_refinement_agent",  
    model="gemini-2.5-flash",
```

```

description=(
    "Evaluates ranked job results and decides whether to exit the loop
or refine the search profile."
),
instruction="""
You will receive:
- user_profile
- ranked_jobs

Look at ranked_jobs.results_quality.

If results_quality is high or medium:
- call the exit_loop tool
- output no text

If results_quality is low:
- refine the user_profile for the next search iteration by doing one or
more of the following:
    - broaden target roles slightly
    - expand related skills
    - loosen location constraints when reasonable
    - increase remote flexibility if appropriate
    - reduce overly restrictive include or exclude preferences if they seem
too narrow

If refining the profile, output ONLY the updated structured profile using
the same keys as user_profile:
- resume_text
- skills
- target_roles
- location
- experience_years
- preferences
""" + instruction_insert,
    tools=[exit_loop],
    output_key="user_profile",
)

resume_critique_agent = LlmAgent(
    name="resume_critique_agent",

```

```
model="gemini-2.5-flash",
description=(
    "Compares the user's resume against the top ranked jobs and gives
tailored resume improvement suggestions."
),
instruction="""
You will receive:
- user_profile
- ranked_jobs
```

Use user\_profile.resume\_text and the top 3 jobs in ranked\_jobs.

For each of the top 3 jobs, generate:

1. missing\_keywords\_or\_skills
2. specific\_bullet\_point\_rewrites\_or\_additions
3. tailored\_resume\_summary\_or\_objective
4. readiness\_score from 1 to 10

For each feedback block include:

- job\_title
- company
- match\_score
- missing\_keywords\_or\_skills
- bullet\_point\_rewrites\_or\_additions
- tailored\_resume\_summary\_or\_objective
- readiness\_score

Keep suggestions specific, professional, and actionable.  
Do not fabricate experience the user does not appear to have.

```
""" + instruction_insert,
    output_key="resume_feedback",
)
```

```
# -----
# COMPOSITE AGENTS
# -----
```

```
parallel_job_search = ParallelAgent(
    name="parallel_job_search",
```

```

        description="Runs LinkedIn, Indeed, and Glassdoor search agents in
parallel.",
        sub_agents=[
            job_search_linkedin,
            job_search_indeed,
            job_search_glassdoor,
        ],
    )

```

```

search_refinement_loop = LoopAgent(
    name="search_refinement_loop",
    description=(
        "Runs job search, job matching, and refinement repeatedly until "
        "results quality is acceptable or max iterations is reached."
    ),
    sub_agents=[
        parallel_job_search,
        job_match_agent,
        search_refinement_agent,
    ],
    max_iterations=3,
)

```

```

full_job_workflow = SequentialAgent(
    name="full_job_workflow",
    description="Runs the full workflow from intake to job search to resume
critique.",
    sub_agents=[
        user_intake_agent,
        search_refinement_loop,
        resume_critique_agent,
    ],
)

```

```

#####
# Root Agents -- SETUP THE ROOT AGENT
#####

```

```

root_agent = full_job_workflow

```

```

#####
# Create the Runner
#####

# Define constants
APP_NAME = "econ249_app"
USER_ID = "user_0"
SESSION_ID = "session_001"

# Create a Session
session = await session_service.create_session(
    app_name=APP_NAME,
    user_id=USER_ID,
    session_id=SESSION_ID,
)

# Create a Runner
runner = Runner(
    agent=root_agent,
    app_name=APP_NAME,
    session_service=session_service
)

#####
# Setup Function to Run the Query
#####
async def call_agent_async(query: str):

    # Prepare the user's message in ADK format
    content = types.Content(role='user', parts=[types.Part(text=query)])

    # Setup the default
    final_response_text = "Agent did not produce a final response." #
    Default

    # Iterate through the response
    print("Internal Process + Eventual Answer (Unformatted):\n")
    async for event in runner.run_async(user_id=USER_ID,
    session_id=SESSION_ID, new_message=content):

```

```

# Print the internal thinking
print("-----")
print("-----")
print(f"    [Event] Author: {event.author}, Type:
{type(event).__name__}, Final: {event.is_final_response()}\n Content:
{event.content}")

# Figure out the final response text
if event.is_final_response():
    if event.content and event.content.parts:
        final_response_text = event.content.parts[0].text
    elif event.actions and event.actions.escalate: # Handle
potential errors/escalations
        final_response_text = f"Agent escalated:
{event.error_message or 'No specific message.'}"

print("\n\nFinal Response:\n")
print(final_response_text)

#####
# QUERY -- ADD QUERY CODE HERE
#####

query = """
I am looking for entry-level or early-career business analyst, consulting,
or strategy roles.

My key skills are:
- Excel
- PowerPoint
- financial analysis
- data analysis
- communication
- teamwork
- presentation skills

My preferred location is Richmond, Virginia, but I am also open to remote
roles.

```

I have 1 year of internship and project experience.

Please help me find relevant jobs and then give me resume feedback based on the strongest matches.

Here is my resume text:

Nathan Potisit

University of Richmond student studying business with experience in finance, consulting, and research.

Experience with Excel, PowerPoint, data analysis, financial analysis, presentations, and teamwork.

Interested in consulting, business analysis, and strategy roles.

"""

```
#####  
# RUN QUERY  
#####  
print(f"User Query: \n{query}\n\n")  
await call_agent_async(query.strip())
```

# Spider AI Assistant



**Assistant Name**

Job Search and Resume Refinement Assistant

**Assistant Description**

A SpiderAI assistant that helps users find relevant jobs based on their skills, interests, and location, then gives tailored resume feedback for the strongest matches.

Assistant Category

**Choose:**

Core

**Search Keywords**

job search; resume feedback; CV improvement; career assistant; business analyst jobs; consulting jobs; strategy jobs

**Assistant Information**

Hi! I'm your Job Search and Resume Refinement Assistant.

I can help you:

- find relevant jobs based on your skills, interests, and location
- identify strong job matches
- compare your resume to job descriptions
- suggest ways to improve your resume for specific roles

To get started, send me:

1. your target job titles or roles
2. your key skills
3. your preferred location or remote preference
4. your years of experience
5. your resume or CV text if you want resume feedback

Instructions

You are a job search and resume refinement assistant.

Your purpose is to help users identify relevant jobs and improve their resume for those jobs.

**Instructions (optional)**

When helping a user:

1. First collect the following information if it is missing:
  - target job titles or roles
  - key skills and technologies
  - preferred location or remote preference
  - years of experience

- industries or companies to include or exclude
- resume or CV text if resume feedback is requested

2. If the user is searching for jobs:

- use the available web search tool to find relevant current job postings
- prioritize relevance to role, skills, location, and experience level
- return structured results including title, company, location, and link
- briefly explain why each job is a match

3. If the user asks for resume feedback:

- compare the resume against the relevant job posting or postings
- identify missing keywords and skills
- suggest stronger bullet points
- suggest a tailored summary or objective
- keep recommendations specific and actionable

4. Be clear, organized, and professional.

5. If information is missing, ask follow-up questions rather than assuming.

6. Do not invent job postings, companies, or links.

7. If no strong matches are found, say so honestly and suggest how to refine the search.

Model

### **Use:**

ChatGPT 5.2

Tool Choices

Turn on:

Google Search / Web Search

## Pt 4: Write Up Sections

## Section 1: Regular Agent + Hypothetical Tool Details

Our agentic workflow is designed to help users with the job search process and resume improvement. It takes in user information, finds relevant jobs, ranks them, and then provides personalized resume feedback.

We created multiple regular agents, each with a specific role:

First, the User Intake Agent collects and organizes the user's information, including their resume, skills, target roles, location, and experience level. This agent is important because it creates a structured profile that all other agents rely on.

Next, we created three job search agents: LinkedIn Search, Indeed Search, and Glassdoor Search. Each of these agents uses a web search tool to find current job postings based on the user's profile. They return job details like title, company, location, and a summary of the role. Using three different platforms helps improve coverage and ensures we don't miss good opportunities.

After that, the Job Match & Rank Agent takes all the job results and evaluates how well each one fits the user. It scores jobs based on skills, role match, location, and experience level, then filters out weaker matches and returns the top results.

The final regular agent is the Resume Critique Agent, which compares the user's resume to the top job matches. It provides specific suggestions, like missing skills, better bullet points, and a tailored summary for each role.

We also included a Search Refinement Agent, which acts as a decision-maker inside the loop. If the job matches are weak, this agent adjusts the search criteria (like broadening roles or relaxing location preferences) and runs the search again.

For tools, we used a web search tool (Google Search). This tool allows the job search agents to pull real, up-to-date job postings from the internet, which is critical for making the assistant useful in real-world scenarios.

## Section 2: Workflow Diagram

User Intake Agent



Search Refinement Loop (max 3 times)

├─ Parallel Job Search

|   ├─ LinkedIn Agent

|   ├─ Indeed Agent

|   └─ Glassdoor Agent



Job Match & Rank Agent



Search Refinement Agent (decides to loop or continue)



Resume Critique Agent



Final Output

This workflow uses multiple design patterns:

- **Sequential flow:** The process follows a clear order from intake → search → ranking → feedback
- **Parallel processing:** Job searches across LinkedIn, Indeed, and Glassdoor happen at the same time
- **Loop structure:** The system can retry the search up to 3 times if results are not strong
- **Coordinator/decision logic:** The refinement agent decides whether to continue or improve the search

### **Section 3: Design Choice Discussion**

We made these design choices to create a workflow that is both realistic and helpful for actual job seekers.

First, we separated the workflow into multiple agents instead of one large agent. This makes the system more organized and easier to debug. Each agent has a clear responsibility, which also improves the quality of outputs.

Second, we used parallel job search agents because job searching in real life involves checking multiple platforms. Running them at the same time makes the process faster and more comprehensive.

Third, we included a loop with refinement because job searches are not always perfect on the first try. Sometimes results are too narrow or not relevant enough. The loop allows the system to adjust and improve results automatically, which makes it feel more intelligent and adaptive.

We also added a ranking system instead of just returning raw job listings. This helps users focus on the best opportunities instead of being overwhelmed with too many options. Finally, the resume critique feature was included to make the workflow more valuable. Instead of just finding jobs, the system helps users actually improve their chances of getting those jobs. This adds a second layer of usefulness and makes the assistant more practical.

Overall, our goal was to design a system that mimics how a real person would approach job searching: gather information, search broadly, refine results, and then improve their application materials.